AHSANULLAH UNIVERSITY OF SCIENCE AND TECHNOLOGY



Department of Computer Science and Engineering

Course No: CSE 2103
Course Title: Data Structures Lab

Assignment No: 04

Date of Submission: 23/2/2026

Submitted by,
Name: Rohan Rahim
Student ID: 00724105101122
Lab Section: C1
Academic Semester: Spring 2025
Program: BSc in CSE

Question:

Implement the below functions of doubly linked list in a single .cpp file.
(1) insert first
(2) insert last
(3) insert anywhere between two nodes (by position, by value)
(4) delete first
(5) delete last
(6) delete from anywhere between two nodes (by position, by value)
(7) printingF() and printingB() -> functions to print the linked list
(8) searching() -> function to search a value in the linked list
(9) last_node() -> function to print the value of the last node
(10) previous_of_last_node() -> function to print the value of the previous node of
last node
(11) list_size() -> function to print the size of the linked list
(12) reversePrint() -> function to print the linked list in reverse order
(13) main function -> no manual node creation/connection is allowed

Solution Code:

```
 #include <bits/stdc++.h>
using namespace std;

struct node {
   int data;
   node* next;
   node* prev;
};

node* root = NULL;
node* tail = NULL;
int listSize = 0;

//1.
void insertFirst(int data) {
   node* temp = new node();
   temp->data = data;

   if (root == NULL) {
       root = tail = temp;
       listSize++;
       return;
```

```
    }

    temp->next = root;
    root->prev = temp;
    root = temp;
    listSize++;
}

//2.
void insertLast(int data) {
    node* temp = new node();
    temp->data = data;

    if (tail == NULL) {
        root = tail = temp;
        listSize++;
        return;
    }

    tail->next = temp;
    temp->prev = tail;
    tail = temp;
    listSize++;
}

//3.
void insertByPosition(int position, int data) {
    int n = listSize;

    if (position < 1 || position > n + 1) {
        cout << "Invalid position.\n";
        return;
    }

    if (position == 1) {
        insertFirst(data);
        return;
    }

    if (position == n + 1) {
        insertLast(data);
        return;
    }

    node* temp = root;
    for (int i = 1; i < position - 1; i++) {
        temp = temp->next;
    }

    node* newNode = new node();
    newNode->data = data;
    newNode->next = temp->next;
```

```
    newNode->prev = temp;
    temp->next->prev = newNode;
    temp->next = newNode;
    listSize++;
}

void insertByValue(int targetValue, int data) {
    if (root == NULL) {
        cout << "DLL is empty.\n";
        return;
    }

    node* temp = root;
    while (temp != NULL && temp->data != targetValue) {
        temp = temp->next;
    }

    if (temp == NULL) {
        cout << "Target value not found.\n";
        return;
    }

    if (temp == tail) {
        insertLast(data);
        return;
    }

    node* newNode = new node();
    newNode->data = data;
    newNode->next = temp->next;
    newNode->prev = temp;
    temp->next->prev = newNode;
    temp->next = newNode;
    listSize++;
}

//4.
void delFirst() {
    if (root == NULL) {
        cout << "DLL is empty.\n";
        return;
    }

    if (root == tail) {
        delete root;
        root = tail = NULL;
        listSize--;
        return;
    }

    node* temp = root;
    root = root->next;
```

```
    root->prev = NULL;
    delete temp;
    listSize--;
}

//5.
void delLast() {
    if (tail == NULL) {
        cout << "DLL is empty.\n";
        return;
    }

    if (root == tail) {
        delete tail;
        root = tail = NULL;
        listSize--;
        return;
    }

    node* temp = tail;
    tail = tail->prev;
    tail->next = NULL;
    delete temp;
    listSize--;
}

//6.
void deleteByPosition(int position) {
    int n = listSize;

    if (position < 1 || position > n) {
        cout << "Invalid position.\n";
        return;
    }

    if (position == 1) {
        delFirst();
        return;
    }

    if (position == n) {
        delLast();
        return;
    }

    node* temp = root;
    for (int i = 1; i < position; i++) {
        temp = temp->next;
    }

    temp->prev->next = temp->next;
    temp->next->prev = temp->prev;
```

```
    delete temp;
    listSize--;
}

void deleteByValue(int value) {
    if (root == NULL) {
        cout << "DLL is empty.\n";
        return;
    }

    node* temp = root;
    while (temp != NULL && temp->data != value) {
        temp = temp->next;
    }

    if (temp == NULL) {
        cout << "Value not found.\n";
        return;
    }

    if (temp == root) {
        delFirst();
        return;
    }

    if (temp == tail) {
        delLast();
        return;
    }

    temp->prev->next = temp->next;
    temp->next->prev = temp->prev;
    delete temp;
    listSize--;
}

//7.
void printingF() {
    if (root == NULL) {
        cout << "DLL is empty.\n";
        return;
    }

    node* temp = root;
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << "\n";
}

void printingB() {
```

```
    if (tail == NULL) {
        cout << "DLL is empty.\n";
        return;
    }

    node* temp = tail;
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->prev;
    }
    cout << "\n";
}

//8.
void searching(int data) {
    node* temp = root;
    while (temp != NULL) {
        if (temp->data == data) {
            cout << "Data found.\n";
            return;
        }
        temp = temp->next;
    }
    cout << "Data not found.\n";
}

//9.
void last_node() {
    if (tail == NULL) {
        cout << "DLL is empty.\n";
        return;
    }
    cout << tail->data << "\n";
}

//10.
void previous_of_last_node() {
    if (tail == NULL || tail->prev == NULL) {
        cout << "Previous of last node does not exist.\n";
        return;
    }
    cout << tail->prev->data << "\n";
}

//11.
void list_size() {
    cout << "The list contains " << listSize << " nodes.\n";
}

//12.
void reversePrint() {
    printingB();
```

```
}

// (13) main: no manual node creation/connection
int main() {
   int choice;

   cout << "\n----- Doubly Linked List Menu -----\n";
   cout << "1. Insert First\n";
   cout << "2. Insert Last\n";
   cout << "3. Insert By Position\n";
   cout << "4. Insert By Value\n";
   cout << "5. Delete First\n";
   cout << "6. Delete Last\n";
   cout << "7. Delete By Position\n";
   cout << "8. Delete By Value\n";
   cout << "9. Print Forward\n";
   cout << "10. Print Backward\n";
   cout << "11. Search Value\n";
   cout << "12. Last Node Value\n";
   cout << "13. Previous of Last Node Value\n";
   cout << "14. List Size\n";
   cout << "15. Reverse Print\n";
   cout << "0. Exit\n";

   while (true) {
       cout << "\nChoose: ";
       cin >> choice;

       if (choice == 0) {
           break;
       }

       int value, position, target;
       switch (choice) {
           case 1:
               cout << "Enter value: ";
               cin >> value;
               insertFirst(value);
               break;
           case 2:
               cout << "Enter value: ";
               cin >> value;
               insertLast(value);
               break;
           case 3:
               cout << "Enter position and value: ";
               cin >> position >> value;
               insertByPosition(position, value);
               break;
           case 4:
               cout << "Enter target value and new value: ";
               cin >> target >> value;
```

```
                insertByValue(target, value);
                break;
            case 5:
                delFirst();
                break;
            case 6:
                delLast();
                break;
            case 7:
                cout << "Enter position: ";
                cin >> position;
                deleteByPosition(position);
                break;
            case 8:
                cout << "Enter value to delete: ";
                cin >> value;
                deleteByValue(value);
                break;
            case 9:
                printingF();
                break;
            case 10:
                printingB();
                break;
            case 11:
                cout << "Enter value to search: ";
                cin >> value;
                searching(value);
                break;
            case 12:
                last_node();
                break;
            case 13:
                previous_of_last_node();
                break;
            case 14:
                list_size();
                break;
            case 15:
                reversePrint();
                break;
            default:
                cout << "Invalid choice.\n";
                break;
        }
    }

    return 0;
}
```

Sample Input Output:

```
----- Doubly Linked List Menu -----
1. Insert First
2. Insert Last
3. Insert By Position
4. Insert By Value
5. Delete First
6. Delete Last
7. Delete By Position
8. Delete By Value
9. Print Forward
10. Print Backward
11. Search Value
12. Last Node Value
13. Previous of Last Node Value
14. List Size
15. Reverse Print
0. Exit

Choose: 1
Enter value: 10

Choose: 2
Enter value: 100

Choose: 3
Enter position and value: 2 20

Choose: 4
Enter target value and new value: 20 30

Choose: 9
10 20 30 100

Choose: 5

Choose: 6

Choose: 9
20 30

Choose: 7
Enter position: 1

Choose: 8
Enter value to delete: 30

Choose: 9
DLL is empty.
```

```
Choose: 1
Enter value: 10

Choose: 2
Enter value: 20

Choose: 2
Enter value: 30

Choose: 2
Enter value: 40

Choose: 10
40 30 20 10

Choose: 11
Enter value to search: 50
Data not found.

Choose: 11
Enter value to search: 30
Data found.

Choose: 12
40

Choose: 13
30

Choose: 14
The list contains 4 nodes.

Choose: 15
40 30 20 10
```